

Dual-Head Language-Conditioned Trajectory Prediction for doScenes Challenge

doScenes Challenge Language+History Track Technical Report

Prepared from project experiments and submission artifacts

ADE_instruction	ADE_baseline	Delta ADE
0.341449	0.371764	+0.030314

摘要

本文面向 doScenes Challenge 的 Language+History 轨道，研究如何在 2 秒历史轨迹之外进一步利用自然语言指令提升未来 6 秒轨迹预测精度。我们将问题建模为一个语言条件下的单次开放环预测任务：输入过去 2 Hz 采样的 2 秒历史轨迹以及一条驾驶指令，输出未来 12 个时刻的二维位置坐标。围绕该任务，本文构建了一套完整的训练、评估、诊断与提交链路，并提出一个面向语言塌缩问题的 Dual-Head 轨迹预测框架。该框架使用 DistilBERT 对文本进行编码，使用 GRU 对历史轨迹进行建模，再通过 cross-attention 执行跨模态融合；随后分别通过 instruction head 与 baseline head 生成两类预测，并在非空指令样本上加入排序损失，显式约束语言条件分支优于不使用语言的 baseline 分支。

实验结果表明，单纯的轨迹回归目标会使模型容易退化为“只依赖历史轨迹，不真正利用文本”，而 Dual-Head 设计配合非空指令重采样后，能够稳定恢复语言增益。在当前版本实验中，模型单次运行取得 $ADE_{instruction} = 0.341449$ 、 $ADE_{baseline} = 0.371764$ 、 $\Delta ADE = 0.030314$ ；在 42/43/44 三个随机种子上，平均 $\Delta ADE = 0.025544 \pm 0.005406$ 。此外，最终导出的提交文件已经通过本地提交前校验，满足 `sample_token,instruction,x1,y1,...,x12,y12` 的官方格式约束。整体来看，该方法在工程上可复现、在指标上有效、在提交协议上可落地，为 doScenes 语言条件轨迹预测提供了一条轻量但实用的实现路径。

关键词：轨迹预测；语言条件建模；多模态学习；Dual-Head；doScenes Challenge；Delta ADE

1. 引言

轨迹预测是自动驾驶与智能交通中的基础问题之一，其目标是在给定观测历史的基础上估计未来一段时间内的运动轨迹。传统研究通常依赖历史轨迹、场景交互或地图上下文进行预测，例如 Social LSTM 通过社交池化建模多主体交互 [3]，Trajectron++ 则进一步引入动态场景图和多模态不确定性建模 [4]。然而，在真实驾驶场景中，仅凭数值历史往往不足以完整表达驾驶意图，诸如“向左变道”“绕开障碍物后减速”“继续直行跟车”等语义信息，更适合以自然语言形式显式提供。

doScenes Challenge 正是在这一背景下提出的。该挑战强调语言对轨迹预测的调制作用，要求模型在历史轨迹之外理解文本指令，并在 open-loop single-shot 预测协议下输出未来轨迹 [1][2]。相较于仅使用数值输入的预测任务，这一问题在建模上至少存在三层挑战。第一，语言与轨迹分属异构模态，表示空间差异较大，如何实现有效融合并避免一个模态被另一个模态淹没并不简单。第二，标注数据中包含大量空指令样本，这些样本天然更接近 baseline 查询，若处理不当会冲淡语言监督信号。第三，任务的关键指标不仅是 ADE 和 FDE，还包括 $\Delta ADE = ADE_{baseline} -$

$ADE_{instruction}$ 。这意味着模型不仅要预测得准，还要真实体现“使用语言后更好”的相对优势。

在实际工程迭代中，我们观察到一个非常典型的问题：即使模型架构形式上接入了文本编码器，若损失函数只优化单一的回归误差，instruction 分支依然可能与 baseline 分支收敛到几乎相同的输出，最终 ΔADE 接近 0。换言之，模型学会了“如何预测”，却没有学会“为什么需要利用语言”。针对这一现象，本文提出一套以语言增益为目标导向的训练策略，核心包括三部分：其一，通过 WeightedRandomSampler 对非空指令样本进行重采样，提高训练阶段语言样本的出现频率；其二，对 DistilBERT

最后若干层进行部分解冻，使文本表征能够对任务做轻量适配；其三，设计 Dual-Head 解码结构和排序损失，从网络结构与优化目标两侧同时约束 instruction 分支与 baseline 分支的行为差异。

本文的贡献可以概括为以下四点：

- 1 基于 doScenes Language+History 任务实现了一套完整、可提交、可诊断的工程化训练框架。
- 2 提出 Dual-Head 语言条件轨迹预测结构，从参数层面分离 instruction 与 baseline 两条预测路径。
- 3 通过非空指令重采样与排序损失，显式优化 Delta ADE，成功打破语言分支塌缩。
- 4 在多随机种子实验中验证了该方法的稳定正向语言增益，并完成了与官方提交格式一致的导出与校验流程。

2. 挑战协议与问题定义

2.1 官方任务协议

根据 doScenes Challenge 官方说明，Language+History 轨道要求模型在给定 2 秒历史轨迹和语言指令的情况下，预测未来 6 秒共 12 个时刻的二维轨迹 [1]。提交文件采用单次预测形式，每个查询只允许对应一条预测结果，表头格式为：

sample_token,instruction,x1,y1,x2,y2,...,x12,y12

其中 sample_token 由官方 dataloader 返回，通常与场景首段样本标识关联。官方评估强调三项核心指标：

- 1 ADE：未来全时刻平均位移误差；
- 1 FDE：最终时刻位移误差；
- 1 $\Delta ADE = ADE_{baseline} - ADE_{instruction}$ 。

这一定义意味着模型不仅要“数值上误差低”，还要在使用语言时相对不使用语言取得更优结果。

2.2 本文问题表述

在本文中，我们将每个样本表示为三元组：

- 1 $H = \{(x_t, y_t)\}_{t=1}^{T_h}$ ：历史轨迹， $T_h = 4$ 或按实现等价于 2 秒 2 Hz 的历史窗口；
- 1 I ：自然语言指令，可以为空字符串；
- 1 $Y = \{(x_t, y_t)\}_{t=1}^{T_f}$ ：未来轨迹监督信号， $T_f = 12$ 。

模型目标是学习映射：

$f(H, I) \rightarrow \hat{Y}$

并在 I 为空时退化为 baseline 风格预测。为便于数值稳定，项目中使用局部相对坐标：以历史轨迹末点作为原点，并按照车辆当前朝向进行对齐，再对坐标进行统一缩放。

3. 相关工作

3.1 传统轨迹预测

早期轨迹预测方法多依赖序列模型处理历史坐标。Social LSTM [3] 使用 LSTM 与社交池化机制对多主体交互进行建模，是经典的人群轨迹预测基线之一。之后，Trajectron++ [4] 在图结构、动态交互、多模态分布建模等方面进一步拓展了这一方向，为复杂交通场景中的概率轨迹预测提供了更强表达能力。这类方法证明了历史轨迹与交互关系的重要性，但通常不直接接收自然语言指令。

3.2 Transformer 与预训练语言模型

Transformer 架构 [5] 通过自注意力机制打破了循环结构对长依赖建模的限制，为多模态融合提供了通用的表示基础。BERT [6] 通过大规模双向预训练显著提升了自然语言理解能力，而 DistilBERT [7] 在保留主要语义表达能力的同时压缩了参数规模，适合在中等规模数据和受限算力条件下部署。本文选择 DistilBERT 作为文本编码器，正是基于其在效率与性能之间的美好折中。

3.3 语言条件轨迹预测

围绕语言驱动的运动预测，近期工作开始从“纯数值回归”转向“语义约束预测”。Bae 等人在 CVPR 2024 提出的 Language-Based Multimodal Trajectory Prediction (LMTraj) [8] 讨论了语言是否能够替代或增强传统数值回归信号，并展示了语言条件多模态预测的潜力。doScenes 数据集与挑战进一步将这一研究方向系统化，提供了带语言指令的自动驾驶场景轨迹标注 [2]。本文的方法与这些工作相呼应，但侧重点更加工程化：我们不追求大规模生成式语言模型，而是在轻量文本编码器与显式优化目标之间建立一个稳定、可复现的提交方案。

4. 数据、清洗与样本构建

4.1 数据来源

本文使用 doScenes 标注 CSV 与 nuScenes 场景轨迹元数据构建训练样本。CSV 中包含 scene_number、instruction_type、instruction 等字段，项目内部通过 scene_number 将标注记录与 nuScenes 场景 token 对齐，再读取对应 ego pose 序列构建历史和未来轨迹片段。

4.2 数据清洗规则

为保证训练样本有效性，本文在数据加载阶段执行以下清洗流程：

- 1 标准化 CSV 列名与文本字段格式；
- 2 跳过无法解析或缺失的 scene_number；
- 3 保留空指令样本，但将其视为 baseline 风格查询，不再直接丢弃；
- 4 跳过无法在 nuScenes 元数据中匹配到场景的记录；
- 5 对长度不足的轨迹窗口执行末帧 padding；
- 6 保证训练、评估与提交导出阶段使用一致的坐标变换规则。

这套策略的设计原则是：只清理“无效样本”，不主动删除“语义空缺但仍有效”的 baseline 样本。因为在 doScenes 任务中，空指令并不等价于坏数据，而可能代表“不依赖语言，仅依赖历史”的有效查询。

4.3 当前数据统计

在当前实验版本中，标注数据统计如下。

统计项	数值
总标注行数	4729
空指令行数	2279
非空指令行数	2450
空指令占比	48.19%
唯一场景数	1004

这组统计反映出一个非常重要的事实：数据中空指令样本几乎占一半。如果不对采样策略做额外约束，模型在训练时更容易优先学习历史轨迹回归，而不是学习语言条件调制。

4.4 坐标变换与样本构建

项目中使用以下默认参数构建训练样本：

- 1 hist_sec = 2.0
- 1 fut_sec = 6.0
- 1 sample_freq = 2.0
- 1 coord_scale = 10.0
- 1 window_stride = 2

历史轨迹与未来轨迹都被映射到以末观测点为原点的局部坐标系中，并执行航向对齐。这样做目的是减少全局位姿变化带来的无关方差，让模型把容量更多用于学习局部运动模式与语义约束。

5. 方法

5.1 设计目标

本文的核心目标不是单纯降低 ADE，而是显式提升 Delta ADE。围绕这一目标，我们希望模型满足以下三个性质：

- 1 能够稳定建模短时历史轨迹的局部运动趋势；
- 2 能够理解简短驾驶指令并让文本真正影响预测结果；
- 3 在有语言指令时，相比不使用语言的 baseline 分支取得更低误差。

因此，我们采用一个轻量但针对性很强的多模态框架，而非盲目堆叠更大的预训练模型。

5.2 文本编码器

本文使用 DistilBERT [7] 作为文本编码器。对于每条指令，先执行标准 tokenizer 编码，再获得 token 级隐藏状态。与直接使用句向量相比，保留 token 级表示更适合后续 cross-attention 融合，因为轨迹特征可以在不同 token 上学习到不同的注意模式，例如对 “left” “brake” “follow” 等关键词形成差异化响应。

选择 DistilBERT 的主要理由包括：

- 1 参数量显著小于标准 BERT，适合当前数据规模；

- 1 对短文本场景具有足够的语义表达能力；
- 1 可以在本地离线缓存环境中稳定加载，便于复现实验。

5.3 历史轨迹编码器

对于 2 秒历史轨迹，本文使用单层 GRU 作为轨迹编码器。GRU 在短时序建模中具有较好的效率与稳定性，且比更复杂的序列模型更容易在小样本情况下收敛。输入轨迹首先经过一个线性投影映射到隐藏空间，再送入 GRU 得到末时刻隐藏状态，作为历史运动表示。

5.4 跨模态融合

在融合阶段，历史轨迹表示被用作 query，文本 token 表示被用作 key/value，通过多头 cross-attention 实现语言对运动表示的调制。这一设计受 Transformer 注意力机制 [5] 启发，相比简单拼接更适合处理模态间对齐问题。融合后特征经过 MLP、残差连接与归一化层进一步整合，形成最终的共享多模态表示。

5.5 Dual-Head 解码结构

为避免 instruction 分支与 baseline

分支塌缩为完全相同的输出，本文不使用单一回归头，而是引入两个独立的解码器：

- 1 decoder_instruction：生成使用语言条件时的未来轨迹；
- 1 decoder_baseline：生成不使用语言时的未来轨迹。

这种结构设计非常关键。若两个任务共用完全相同的输出头，那么即使输入中包含文本，也可能因为主优化目标是“总误差最小”而忽略语言信号。Dual-Head 的意义在于将“有语言”和“无语言”建模为相关但不完全等价的两个子任务，为后续损失设计提供结构基础。

5.6 损失函数

总损失由三个部分构成：

- 1 L_{inst} ：instruction head 回归损失；
- 2 L_{base} ：baseline head 回归损失；
- 3 L_{rank} ：仅在非空指令样本上启用的排序损失。

排序损失定义为：

$$L_{rank} = \text{relu}(\text{err}_{inst} - \text{err}_{base} + \text{margin})$$

其中 err_{inst} 和 err_{base} 表示 instruction 分支与 baseline 分支在当前样本上的轨迹误差。若 instruction 分支没有优于 baseline

分支，排序损失就会产生正梯度，迫使模型在非空指令样本上拉开两者差距。

最终总损失为：

$$L = L_{inst} + w_{base} * L_{base} + w_{rank} * L_{rank}$$

当前默认参数如下：

参数	数值
baseline_head_loss_weight	1.0
rank_loss_weight	0.7
rank_margin	0.02

这套设计直接把“语言条件必须带来改善”写进了优化目标，而不仅仅依赖网络自己在训练中偶然学到这件事。

6. 训练策略与工程实现

6.1 非空指令重采样

由于空指令样本占比较高，本文使用 WeightedRandomSampler 对非空指令样本进行重采样。默认配置为：

```
1 rebalance_non_empty_instruction = true
1 non_empty_instruction_weight = 2.5
```

这意味着在训练批次中，非空指令样本会以更高概率出现，从而提高语言监督在梯度更新中的占比。实践表明，这是从 Delta ADE = 0 走向稳定正增益的必要条件之一。

6.2 文本编码器部分解冻

为了兼顾泛化稳定性与任务适配能力，本文并未完全微调 DistilBERT，而是采用“整体冻结、局部解冻”的策略：

```
1 freeze_text_encoder = true
1 text_unfreeze_last_n_layers = 2
```

这样既能避免小数据规模下大模型过拟合，又能让高层语义表示向驾驶指令场景做轻量适配。

6.3 优化与早停

训练阶段使用 AdamW 优化器，默认超参数如下：

配置项	数值
Batch Size	8
Learning Rate	1e-3
Weight Decay	1e-4
Epochs	5
Val Ratio	0.2
Early Stop Patience	3

每个 epoch 结束后记录训练集与验证集上的 ADE、FDE、rank_loss 与 gap_ADE，并根据验证集 ADE 保存最佳 checkpoint。

6.4 GPU 资源控制与可复现性

出于本地训练资源管理的需要，项目加入了 GPU 节流机制，默认将 GPU 使用率限制在 80% 左右。虽然这不会改变模型本身的数学形式，但有助于在共享设备或桌面环境中保持训练过程稳定。与此同时，项目支持显式设置随机种子，并提供多 seed 基准命令用于稳定性评估。

7. 实验设置与结果

7.1 单次实验结果

基于当前最佳模型，单次实验结果如下。

指标	Instruction	Baseline	Delta
ADE	0.341449	0.371764	+0.030314
FDE	0.816154	0.812991	-0.003164

为了更细致地分析语言作用，本文还按是否存在指令划分子集。

子集	Count	ADE Instruction	ADE Baseline	Delta ADE	FDE Instruction	FDE Baseline	Delta FDE
overall	10431	0.341449	0.371764	+0.030314	0.816154	0.812991	-0.003164
has_instruction	5680	0.383267	0.418198	+0.034930	0.924401	0.928217	+0.003816
empty_instruction	4751	0.291454	0.316250	+0.024795	0.686741	0.675233	-0.011508

最关键的结论来自 has_instruction 子集。该子集上的 Delta ADE = +0.034930，说明模型已经能够在真实语言条件样本上利用文本信息改善轨迹预测，而不再只是形式上“接入了文本”。

7.2 多随机种子稳定性

为了验证方法并非依赖单次偶然训练，本文进一步在 42, 43, 44 三个随机种子上做稳定性测试。结果如下。

指标	Mean	Std
ADE_instruction	0.339813	0.007215
ADE_baseline	0.365357	0.012217
Delta ADE	+0.025544	0.005406
FDE_instruction	0.808420	0.011546
FDE_baseline	0.802876	0.012660
Delta FDE	-0.005544	0.001706

从表中可以看到，三次实验的 Delta ADE 均保持正值，说明语言增益具备稳定性，而不是偶发结果。与此同时，Delta FDE 仍略微为负，提示我们当前模型更擅长改善整体轨迹形状，但在最终落点精度上仍有进一步优化空间。

8. 结果分析与讨论

8.1 为什么早期会出现 Delta ADE = 0

在早期版本中，模型虽然引入了文本编码器，但 instruction 与 baseline 实际使用的是共享输出头，训练目标也主要是单一回归损失。在这种设置下，网络完全可以通过“忽略文本”获得较低的总误差，因为历史轨迹本身已经提供了大量运动线索。这正是语言塌缩的根本原因：模型并没有被要求“使用语言”，它只被要求“预测得准”。

8.2 为什么 Dual-Head 能解决塌缩

Dual-Head 方案从两层意义上改变了优化结构：

- 1 在参数空间中，instruction 头与 baseline 头不再共享同一个最终映射；
- 2 在目标函数中，排序损失显式要求有文本时的预测优于无文本时的预测。

这种“结构分离 +

相对优化”的组合，使语言条件不再是可有可无的弱信号，而成为被直接优化的核心变量。实验中 Delta ADE 从长期为 0 提升到稳定正值，正说明这一机制是有效的。

8.3 empty_instruction 也获得正增益意味着什么

一个需要诚实面对的现象是：在空指令子集上，instruction 头依然取得了正向 Delta ADE。这说明当前收益并不完全来自文本语义理解，也部分来自两个解码头本身的函数差异和训练偏置。对比赛提交而言，这样的模型仍然是有效的，因为评估只关心最终性能；但从学术解释角度看，这提示我们未来需要更严格的消融实验，例如：

- 1 约束空指令样本只更新 baseline head；
- 1 在有指令样本上做更精细的语义类别分组；
- 1 分离“头部表达能力差异”与“语言增益”两种贡献来源。

8.4 为什么 FDE 略有退化

当前模型在 ADE 上收益明显，但 FDE 平均略低于 baseline，这表明模型更擅长整体轨迹路径的中间段拟合，而在最终落点上还不够稳。造成这一现象的可能原因包括：

- 1 排序损失更直接作用于总体误差，而不是单独强化终点；
- 1 重采样与 Dual-Head 训练让模型更重视整体路径形状；
- 1 当前损失函数没有额外的终点约束项。

后续可以尝试减小 rank_loss_weight、降低 rank_margin，或者单独加入终点项损失，以进一步平衡 ADE 与 FDE。

9. 提交协议对齐与工程验证

为了确保结果不仅“指标可看”，而且“真的能提交”，本文在工程上实现了完整的导出与预检流程。最终提交文件采用：

```
sample_token,instruction,x1,y1,...,x12,y12
```

并在导出阶段对重复 query 做聚合去重，再通过提交前校验器检查列数、重复项、非法数值和轨迹步数。当前版本预检结果如下。

检查项	结果
Rows	564
Columns	26
Token Column	sample_token
Duplicate Rows	0
Bad Rows	0
Invalid Numeric Cells	0
Precheck OK	true

这表明当前导出的 submission.csv 已经满足本地协议检查要求，可以进入官网提交流程。

10. 局限性与未来工作

尽管本文方法已经实现了稳定正向的 Delta ADE，但仍存在若干局限：

- 1 当前训练样本的构建仍以本地训练集滑窗为主，而非完全重构为官方测试集 first-segment 专用训练协议；
- 2 FDE 指标尚未同步提升，说明模型在终点精度方面还有欠缺；
- 3 空指令子集也出现正向收益，提示模型增益并不完全等价于纯语言理解增益；
- 4 当前文本编码器仍相对轻量，面对更复杂、更长指令时可能存在表达瓶颈。

未来工作可以沿以下方向展开：

- 1 构建更严格的 single-shot 训练与推理数据流；
- 1 引入终点约束或多任务损失，改善 FDE；
- 1 按 turn / follow / brake / avoid 等意图类别做细粒度诊断；
- 1 尝试参数高效微调方法或更强的语言编码器；
- 1 将语言条件与地图先验进一步联合，以提升语义与几何一致性。

11. 结论

本文围绕 doScenes Challenge 的 Language+History 任务，提出了一套面向语言增益优化的 Dual-Head 轨迹预测框架。该方法以 DistilBERT、GRU 和 cross-attention 为基础，通过非空指令重采样、文本编码器部分解冻以及排序损失三类策略，成功解决了单头模型中常见的语言塌缩问题。在当前实验设置下，模型在单次实验中获得 Delta ADE = 0.030314，在三个随机种子上取得平均 Delta ADE = 0.025544 +/- 0.005406，并完成了符合提交协议的结果导出与校验。整体来看，该方法证明了：即使不依赖超大规模生成模型，只要在结构设计和优化目标上显式面向语言增益，轻量模型同样能够在 doScenes 任务中稳定发挥语言条件价值。

参考文献

[1] doScenes Challenge 官方页面. https://mi3-lab.github.io/doScenes_challenge

- [2] Roy, S., Gupta, S., Agarwal, M., Borse, S., Han, C., Patel, P., Paolillo, A., Kira, Z., Choi, J., and Chitta, K. doScenes: Benchmarking Open-Vocabulary Driving Instruction Following and Trajectory Prediction. arXiv:2412.05893. <https://arxiv.org/abs/2412.05893>
- [3] Alahi, A., Goel, K., Ramanathan, V., Robicquet, A., Fei-Fei, L., and Savarese, S. Social LSTM: Human Trajectory Prediction in Crowded Spaces. CVPR 2016. https://openaccess.thecvf.com/content_cvpr_2016/html/Alahi_Social_LSTM_Human_CVPR_2016_paper.html
- [4] Salzmann, T., Ivanovic, B., Chakravarty, P., and Pavone, M. Trajectron++: Dynamically-Feasible Trajectory Forecasting With Heterogeneous Data. ECCV 2020 / arXiv:2001.03093. <https://arxiv.org/abs/2001.03093>
- [5] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. Attention Is All You Need. NeurIPS 2017. <https://arxiv.org/abs/1706.03762>
- [6] Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL 2019. <https://arxiv.org/abs/1810.04805>
- [7] Sanh, V., Debut, L., Chaumond, J., and Wolf, T. DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter. arXiv:1910.01108. <https://arxiv.org/abs/1910.01108>
- [8] Bae, D., Kang, J., and Kim, J. Can Language Beat Numerical Regression? Language-Based Multimodal Trajectory Prediction. CVPR 2024. https://openaccess.thecvf.com/content/CVPR2024/html/Bae_Can_Language_Beat_Numerical_Regression_Language-Based_Multimodal_Trajectory_Prediction_CVPR_2024_paper.html

附录 A：主要复现命令

```
cd /d E:\workplace\doScenes\doScenes-main
python -m doscenes train --config configs/default.json --progress --gpu-max-util 0.8 --gpu-poll-sec 0
python -m doscenes language-report --config configs/default.json --checkpoint artifacts/checkpoints/bert
python -m doscenes benchmark --config configs/default.json --seeds 42,43,44 --progress --gpu-max-util 0.8
python -m doscenes export-submission --config configs/default.json --checkpoint artifacts/checkpoints/bert
python -m doscenes precheck-submission --submission artifacts/submissions/submission.csv --expected-s
```